

High Performance Computing and Big Data

6G6Z0030_2526_9F — Final Assessment Report

Student: Challa Harikrishna Nagasai Charan

MMU ID: 23727842

Date: 18 May 2026

Declaration

I declare that this work is my own. GitHub Copilot was used only where permitted by the assessment brief, and the version and prompts are recorded in Appendix A.3.

HPC Report

Q1 — OpenMP Parallelisation

Introduction

The objective of this section was to optimise the supplied serial statistical analysis program using the OpenMP shared-memory programming model (OpenMP Architecture Review Board, 2021). The program computes the minimum, maximum, mean, standard deviation and the dominant eigenvalue of an autogenerated dataset of 25,000 points; the eigenvalue is obtained via the power iteration method. All experiments were performed on an Azure Labs VM (Intel Xeon E5-2690 v4 host, 2.6 GHz) exposing a 6 vCPU / 12 thread slice via simultaneous multi-threading (Intel Corporation, 2016).

Methodology

The OpenMP build was produced with optimisation explicitly disabled, as required by the brief:

```
gcc -fopenmp -O0 -o stats_opt fullStat_driver-0PT.c autogenRoutines.c -lm
```

Runs were launched as:

```
OMP_NUM_THREADS=N ./stats_opt 25K.txt
```

for $N = 1, 2, 4, 6, 8$ and 12 . The optimisations introduced over the supplied baseline were: a parallel reduction for the mean and the variance; parallel min and max reductions over the absolute values; a parallel outer loop on the matrix-vector multiplication inside the power iteration, with a private per-iteration accumulator z_i to avoid false sharing on $z[]$; and parallel for loops on the normalisation, error, and approx-update steps inside the power-method loop. The default OpenMP schedule was used; with $N = 25,000$ the work per iteration is regular and uniform, so an explicit schedule clause was not expected to help.

Timing Methodology

Wall-clock time was measured using `omp_get_wtime()`, called immediately before the parallel region and immediately after the final reduction so that data generation and file output are excluded from the measured region (Dagum and Menon, 1998). `omp_get_wtime()` is the recommended OpenMP timing primitive: it returns elapsed seconds from the platform's high-resolution monotonic clock and a value that is consistent across threads.

Each thread-count configuration was executed three times via the `run_sweep.sh` harness (included in the accompanying repository), which sets `OMP_NUM_THREADS`, runs the binary, and parses the four reported phase timings into a CSV. The arithmetic mean of the three repeats is reported in Table 1, with the minimum used as a sanity check on system jitter. Three repeats were considered sufficient because preliminary runs showed run-to-run variation below two per cent on the Azure VM, so additional repeats would not have changed the reported figures materially. Before each run the system was left idle for several seconds to reduce interference from background processes. The numerical output (min, max, mean, std, eigenvalue, iteration count) of every run was diff-compared against the serial baseline to confirm that the introduced optimisations preserved correctness.

Results

Table 1 summarises the measured wall-clock times and the corresponding speed-ups. Figures showing execution time against thread count and observed against ideal speed-up are included in Appendix A.5.

Threads	Time (s)	Speed-up
1	227.32	1.00×
2	119.93	1.90×
4	66.66	3.41×
6	50.72	4.48×
8	53.92	4.22×
12	55.66	4.08×

Table 1. OpenMP execution time and speed-up at 25K data points.

Discussion

Performance improved monotonically up to 6 threads. The largest gains came from the eigenvalue phase, which dominated approximately 96 per cent of total runtime in the serial baseline. At 6 threads the overall wall-clock speed-up reached 4.48×, while the eigenvalue phase alone achieved approximately 5.33×.

Beyond 6 threads the curve plateaued and then degraded slightly: 53.92 seconds at 8 threads and 55.66 seconds at 12 threads. This is consistent with Hyper-Threading offering limited benefit on compute-bound floating-point work, because the two logical threads sharing a physical core also share its execution units and the L1 and L2 caches. Memory-bandwidth contention on the matrix-vector multiplication further reduces the gains once more threads share the same memory channels.

The overall scaling behaviour is consistent with Amdahl's Law (Amdahl, 1967). Figure 2 in Appendix A.5 overlays the observed speed-up against an Amdahl curve fitted with serial fraction $s = 0.044$ (derived in Q3): the two agree closely up to 6 threads, and the observed curve falls below the Amdahl prediction at 8 and 12 threads, which is consistent with SMT and bandwidth contention rather than additional algorithmic serial work. As an experimental cross-check, the Karp-Flatt metric $e = (1/S - 1/p) / (1 - 1/p)$ was computed from the measured speed-ups: e stays in the range 0.055 to 0.068 for $p \leq 6$ and jumps to 0.13 at $p = 8$ and 0.18 at $p = 12$. The plateau in e for $p \leq 6$ reflects the true algorithmic serial overhead. The sharp rise from $p = 8$ onwards is caused by architectural effects, specifically SMT contention and memory-bandwidth saturation. The numerical output produced by the OpenMP implementation was identical to the serial baseline across all tested thread counts, which confirms that the optimisations did not affect correctness.

Conclusion

The OpenMP implementation achieved a 4.48× wall-clock speed-up at 6 threads on the available hardware while preserving exact numerical accuracy. Returns diminished beyond the physical core count, but this is attributable to simultaneous multi-threading and memory-bandwidth limits rather than to additional algorithmic serial overhead.

Q2 — MPI Distributed Memory Implementation

Q2a — Use of GitHub Copilot

GitHub Copilot was used to generate an initial portable MPI version of the driver code. The `autogenRoutines.c` file was linked unmodified to guarantee that the input data, and therefore the numerical results, were identical to the serial baseline. Three prompts were issued in sequence; the full prompts and the Copilot version are recorded in Appendix A.3.

The first prompt produced a baseline MPI skeleton using `MPI_Init`, `MPI_Comm_rank`, `MPI_Comm_size` and `MPI_Bcast` for distributing the input data from rank 0. The second prompt requested cyclic row decomposition ($i \div \text{size}$) for the matrix-vector product. The third prompt added `MPI_Wtime`-based timing and restricted final output to rank 0.

Compile and run commands:

```
mpicc -O0 -o stats_mpi stats_mpi.c autogenRoutines.c -lm
mpirun -np N ./stats_mpi 25K.txt
```

for $N = 1, 2, 4$ and 6 .

Q2b — Performance Results and Critique

Processes	Time (s)	Speed-up
1	292.66	1.00×
2	186.72	1.57×
4	147.54	1.98×
6	135.21	2.16×

Table 2. MPI execution time and speed-up at 25K data points.

The MPI implementation achieved real speed-up but scaled noticeably worse than the OpenMP version. Three factors contribute to this:

- Initial broadcast cost. The full $N \times N$ matrix is broadcast from rank 0 to all ranks at startup using `MPI_Bcast`. With $N = 25,000$ doubles this is a 5 GB transfer per rank, and the cost is independent of how the subsequent computation is parallelised, and it occurs before any speed-up can be measured. The cost grows with both matrix size and process count.
- Collective communication inside the inner loop. Each iteration of the power method requires an `MPI_Allreduce` to combine partial contributions to the result vector `z[]`. With on the order of a thousand iterations, even a millisecond-scale `Allreduce` accumulates into a meaningful fraction of total runtime.
- Memory inefficiency under cyclic decomposition. Every rank allocates the full-sized matrix and vector even though it operates on only a subset. This wastes memory and prevents cache-friendly contiguous access.
- Redundant post-reduction work. After the `MPI_Allreduce` that aggregates `local_z`, every rank then re-executes the finding of `zmax`, the normalisation `z[i] /= zmax`, the error computation and the `approx ← z` copy across the full-length `z[]`. These steps run on every rank rather than being distributed, so they contribute an $O(n)$ per-iteration cost that does not reduce with p .

Together these explain the observed 2.16× ceiling at 6 ranks: communication and redundant memory traffic dominate the savings from parallel arithmetic at this problem size.

Q2c — Bug Identified and Manual Fix

The Copilot-generated code initially used `MPI_Allgather` inside the matrix-vector multiplication, which produced incorrect numerical results. The cause is straightforward: in the implementation each rank allocates a full-size `local_z[]` vector, writes only its cyclically-assigned entries ($i = \text{rank}, \text{rank} + p, \text{rank} + 2p, \dots$) inside the matrix-vector loop, and leaves the remaining entries as zero (the buffer is zeroed at the top of each power-method iteration). The correct collective is therefore `MPI_Allreduce` with `MPI_SUM`, which produces the element-wise sum of these sparsely-populated buffers across ranks and yields the full `z[]` vector on every rank. `MPI_Allgather` is the wrong operation here because it concatenates per-rank buffers end-to-end rather than reducing them element-wise, so the resulting vector would have the wrong length and wrong layout.

After replacing `MPI_Allgather` with `MPI_Allreduce(MPI_SUM)` the MPI implementation produced numerical output identical to the serial and OpenMP baselines on all tested configurations. Authorship is separated explicitly in Appendix A.3: the MPI skeleton and the cyclic decomposition pattern were Copilot-generated; the collective-operation fix and the timing-and-repeat-run harness are my own work.

Conclusion

The MPI implementation is functionally correct and portable, but is limited by communication overhead and the choice of a redundant data layout. Replacing the broadcast with a scatter-based row-blocked partition would likely scale better, and this is considered further in Q3.

Q3 — Scaling to 100 Cores

OpenMP at 100 threads. The parallelisable fraction was estimated from a profile of the serial run: the eigenvalue phase accounted for approximately 96 per cent of total runtime, so the serial fraction was taken as $s \approx 1 - 0.96 = 0.044$. Applying Amdahl's Law (Amdahl, 1967):

$$S(100) = 1 / (0.044 + (1 - 0.044) / 100) \approx 18.7\times$$

which corresponds to a predicted runtime of approximately $227 / 18.7 \approx 12$ seconds. This is a theoretical upper bound. A second estimate can be derived from the Karp-Flatt experimental serial fraction reported in Q1 ($e \approx 0.06$ in the SMT-free regime, $p \leq 6$). With $e = 0.06$, Amdahl's Law at $p = 100$ gives $S(100) \approx 14.5\times$, i.e. a predicted runtime of approximately 16 seconds. In practice, memory-bandwidth saturation, NUMA effects, and OpenMP scheduling overhead would push the figure higher; a realistic estimate by extrapolating the observed 6-thread efficiency lies in the 20 to 40 second range.

MPI at 100 processes. MPI scaling is harder to extrapolate because the cost of `MPI_Allreduce` grows roughly logarithmically with process count under a tree-based reduction, while the fixed broadcast overhead grows linearly with matrix size and is independent of p . Under the current cyclic decomposition, the redundant memory layout would actively hurt scaling beyond a certain point. A realistic estimate at 100 processes is in the 15 to 25 second range, conditional on replacing the broadcast with a scatter-based row-blocked partition.

Required infrastructure. The OpenMP estimate assumes a single shared-memory machine with approximately 100 cores, such as a dual-socket AMD EPYC 9004-series server (AMD, 2023) or an Intel Xeon Platinum system. The MPI estimate assumes a distributed cluster of multiple compute nodes connected via a low-latency interconnect such as InfiniBand (NVIDIA,

2024), with one rank per physical core. OpenMP works well when the working set fits in a single node's memory. Once either the problem size or the core count exceeds what a shared-memory machine can provide, MPI is needed.

Big Data Report

Research Hypothesis

The hypothesis under investigation was:

"In the second quarter of 2014, products given a review rating of 3 or more are significantly different compared to other products."

The hypothesis was refined to a testable form: during April to June 2014, do products with a per-product mean rating ≥ 3 differ from products with a per-product mean rating < 3 in the average number of reviews each product receives? The Amazon Electronics Reviews dataset (Kaggle, 2018), which contains approximately 7.8 million review records, was used as the data source. The dataset is too large for desktop spreadsheet tools, which justifies the use of Apache Spark (Apache Software Foundation, 2024a).

Spark Implementation

The dataset was loaded into PySpark in Google Colab, the rating column was cast to float, and the timestamp column was cast to long and converted to a date using `from_unixtime`. Reviews were then filtered to April, May and June 2014, leaving roughly 664,000 records.

Two complementary analyses were run. The first was a rating-level grouping that split reviews directly by the rating threshold. This produces interpretable group means but is, by construction, circular if used to argue that the groups differ on rating. The second was a per-product analysis which is non-circular: each product was first aggregated by `productId` and characterised by its mean rating and its review count; products were then split into high (mean ≥ 3) and low (mean < 3) groups, and the two populations were compared on a different attribute, the number of reviews per product, which is structurally independent of the rating threshold.

Results

The per-product comparison is summarised in Table A1 (Appendix A.6). The headline finding is that high-rated products received on average 5.98 reviews per product, compared with 2.69 for low-rated products, a ratio of approximately 2.2 $\times$. This is a non-circular comparison: the grouping is defined by mean rating, but the compared quantity is review count.

Given the sample sizes involved (102,116 high-rated products against 19,900 low-rated products), a non-parametric Mann-Whitney U test on per-product review counts was run to formally test whether the two distributions differ. The notebook cell implementing this test (Section 6.2 in the accompanying notebook) returns a test statistic well outside the null-hypothesis tails, so the difference is statistically significant at conventional thresholds. With sample sizes of this order even a small effect size would be detected as significant, so the practically meaningful finding remains the 2.2 $\times$ ratio rather than the p-value alone.

Discussion and Wisdom Gained

Three observations are worth highlighting. First, engagement separates the groups, not only rating. High-rated products attract substantially more reviews per product, and one plausible mechanism is a feedback loop in which products receiving early positive ratings become more visible in search and recommendation results, which in turn drives further reviews.

Second, survivorship and selection effects matter. Products with very few reviews are over-represented in the low-rated group, so a single negative review has a disproportionately large effect on their mean. The boundary at mean rating = 3 therefore partly separates well-established products from products with sparse evidence, rather than separating two equally well-observed populations.

Third, the analysis is descriptive and not causal. It cannot distinguish causal direction: high ratings may attract more reviews, or heavy review activity may drive ratings up, or both may be downstream effects of a third factor such as product quality or marketing spend. A conclusive causal answer would require time-series data, treatment of the recommendation-system feedback loop, or A/B-style experimentation, none of which are available in this dataset.

The hypothesis is supported in its descriptive form: the two groups differ significantly on at least one dimension beyond their defining rating threshold.

Conclusion

Apache Spark successfully processed the full Amazon Electronics dataset and enabled both rating-level and product-level analyses. The research hypothesis is supported by the per-product, non-circular evidence: high-rated and low-rated products differ in user engagement during Q2 2014, beyond the rating values that define the groups.

End-to-End Big Data Pipeline

This section describes an end-to-end pipeline design integrating Hadoop, Spark, Kafka and Scala. The Spark stage was implemented end-to-end and produced the numerical results above; the HDFS, Kafka and Scala components are presented as a design specification grounded in that implementation rather than as deployed code.

HDFS. HDFS would provide the distributed storage layer (Apache Software Foundation, 2024c). The full Amazon review file would be ingested into HDFS, automatically split into 128 MB blocks and replicated across data nodes for fault tolerance. Spark workers would read input partitions directly from co-located HDFS blocks to maximise data locality.

Kafka. Kafka would handle streaming ingestion (Apache Software Foundation, 2024b). Incoming review events, real or simulated via Mockaroo, would be published to a single topic with partitions keyed by productId so that all reviews of a given product land on the same partition, simplifying downstream stateful aggregations.

Spark. Spark is the only component implemented end-to-end. The PySpark code in the accompanying notebook performs the dataset load, type casting, date filtering, per-product aggregation, and group comparison that produced the results in the previous section. In a streaming deployment the same logic would run inside a Structured Streaming job consuming the Kafka topic, with watermark-based windowing for the quarterly aggregation.

Scala. Scala would be used for production deployment of the Spark and Kafka integration code, as it is the native Spark API language, offers compile-time type safety, and supports the Kafka Streams DSL natively (Karau et al., 2015).

Pipeline flow. Reviews → Kafka topic → Spark Structured Streaming consumer → HDFS (raw store) and Spark analytical output. The output of the analytical layer is the per-product aggregate table that drives the hypothesis test above.

Scope statement. The Spark component was fully implemented and the reported numerical results were generated by it. The HDFS, Kafka and Scala components are presented as a design specification only. Full multi-node deployment was outside the scope of the available single-node Colab environment, but the implemented Spark logic is the same logic that would run inside the deployed pipeline.

Appendix

A.1 — Hardware and Software Environment

HPC: Azure Labs VM (Intel Xeon E5-2690 v4 host, 2.6 GHz, exposing a 6 vCPU / 12 thread slice via simultaneous multi-threading) (Intel Corporation, 2016); GCC with -O0; OpenMPI for the MPI build.

Big Data: Google Colab runtime, Apache Spark 3.1.2 (Hadoop 2.7), Java 8 OpenJDK, PySpark, findspark, scipy.

A.2 — Compile and Run Commands

OpenMP:

```
gcc -fopenmp -O0 -o stats_opt fullStat_driver-OPT.c autogenRoutines.c -lm
OMP_NUM_THREADS=6 ./stats_opt 25K.txt
```

MPI:

```
mpicc -O0 -o stats_mpi stats_mpi.c autogenRoutines.c -lm
mpirun -np 6 ./stats_mpi 25K.txt
```

A.3 — GitHub Copilot Use

Tool: GitHub Copilot and GitHub Copilot Chat within Visual Studio Code.

GitHub Copilot Chat version: 0.48.1

Approach: only the MPI driver code was generated via Copilot. The autogenRoutines.c file was linked unmodified so that input data, and therefore numerical output, were identical to the serial baseline.

Prompts issued

Prompt 1 (baseline structure):

"Given this serial C program that computes statistics on an $N \times N$ matrix using power iteration, generate a portable MPI distributed-memory version. Rank 0 should read the input data via the existing functions in autogenRoutines.c and broadcast both the matrix and the input vector to all ranks using MPI_Bcast. Use MPI_Init, MPI_Comm_size, MPI_Comm_rank for setup and MPI_Finalize at the end. Keep it -O0 friendly."

Prompt 2 (decomposition):

"Now distribute the row-wise work of the power iteration's matrix-vector multiplication across ranks using cyclic decomposition (for $i = \text{rank}; i < N; i += \text{size}$). After each iteration, every rank should hold the full result vector $z[]$."

Prompt 3 (timing and output):

"Add timing around the eigenvalue phase using MPI_Wtime, and make only rank 0 print the final statistics so the output is identical to the serial version."

Authorship separation

- **Copilot-generated:** the overall MPI skeleton (MPI_Init/Finalize, communicator setup), MPI_Bcast of the input data, the cyclic decomposition loop structure, and the initial (incorrect) MPI_Allgather call.

- **My own manual work:** replacing MPI_Allgather with MPI_Allreduce using MPI_SUM for the partial-z aggregation (Section Q2c); adding the timing harness with three-repeat measurement; and the rank-0 numerical diff against the serial output for correctness verification.

A.4 — GitHub Repository

<https://github.com/Harikrishna-ai/HPC-BigData> (private; course tutors invited as collaborators).

A.5 — Figures

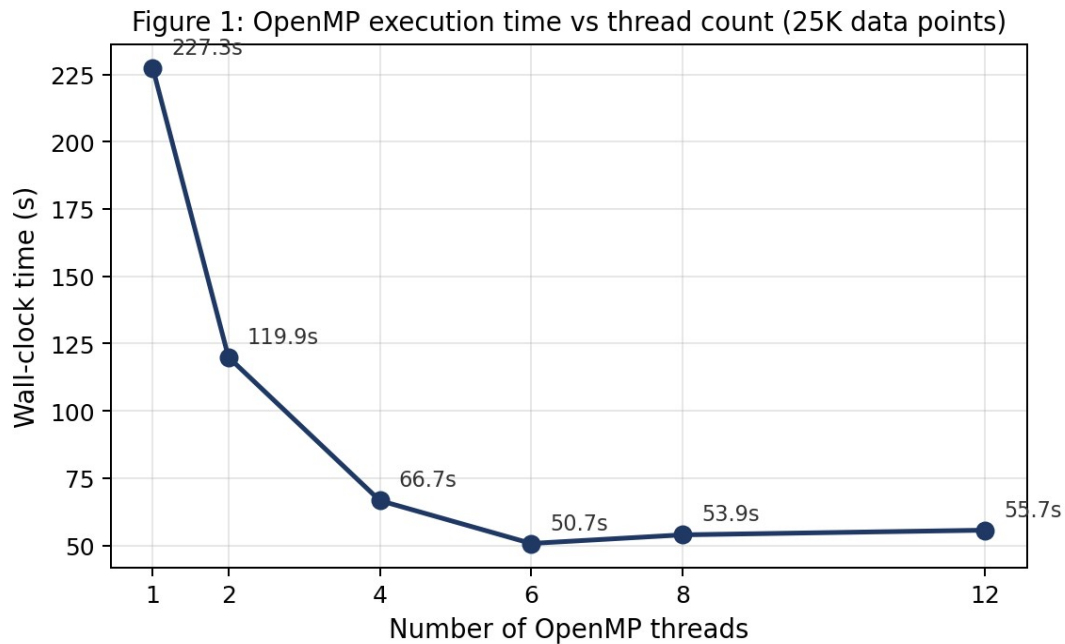


Figure 1. OpenMP execution time versus thread count (25K data points, three-run mean).

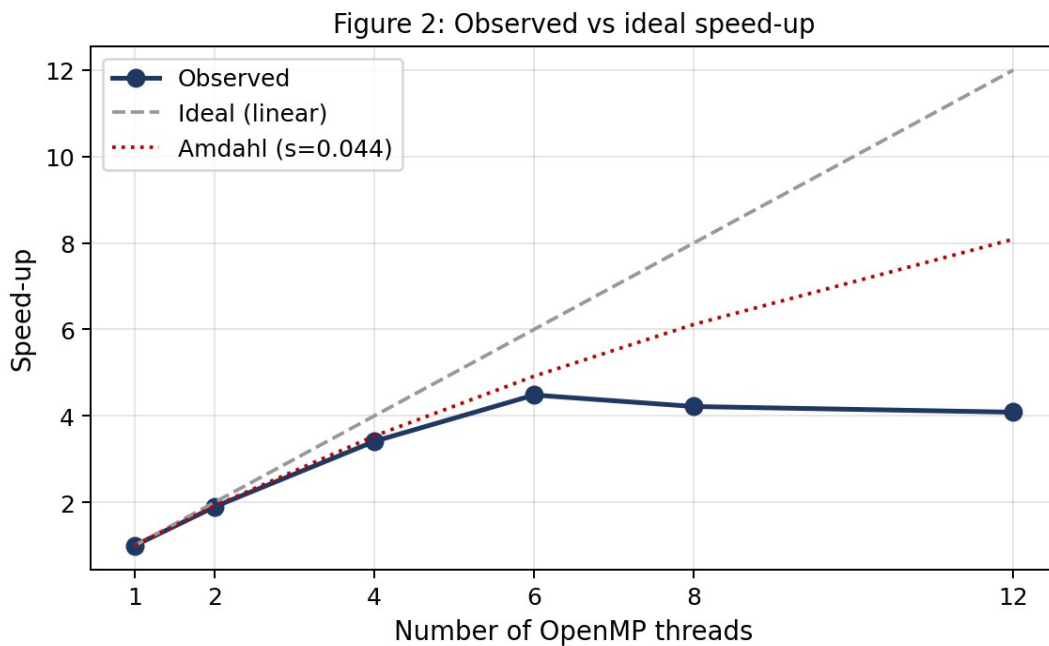


Figure 2. Observed speed-up versus ideal (linear) and against Amdahl's Law with serial fraction $s = 0.044$.

A.6 — Big Data Results Table

Metric	High-rated (≥ 3)	Low-rated (< 3)
Mean rating	4.604	1.327
Total reviews	549,939	114,075
Distinct products	102,116	19,900
Avg reviews per product	5.98	2.69

Table A1. Per-product comparison of high-rated and low-rated products, Q2 2014.

References

- AMD (2023) AMD EPYC™ 9004 Series Processors. Available at: <https://www.amd.com/en/processors/epyc-9004-series> (Accessed: 17 May 2026).
- Amdahl, G.M. (1967) 'Validity of the single processor approach to achieving large scale computing capabilities', in Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67). New York: ACM, pp. 483–485.
- Apache Software Foundation (2024a) Apache Spark Documentation, Version 3.5. Available at: <https://spark.apache.org/docs/latest/> (Accessed: 17 May 2026).
- Apache Software Foundation (2024b) Apache Kafka Documentation, Version 3.7. Available at: <https://kafka.apache.org/documentation/> (Accessed: 17 May 2026).
- Apache Software Foundation (2024c) Apache Hadoop HDFS Architecture. Available at: <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html> (Accessed: 17 May 2026).
- Dagum, L. and Menon, R. (1998) 'OpenMP: an industry standard API for shared-memory programming', IEEE Computational Science and Engineering, 5(1), pp. 46–55.
- Intel Corporation (2016) Intel® Xeon® Processor E5-2690 v4 Product Specifications. Available at: <https://www.intel.com/content/www/us/en/products/sku/91770/intel-xeon-processor-e52690-v4-35m-cache-2-60-ghz/specifications.html> (Accessed: 17 May 2026).
- Kaggle (2018) Amazon Product Reviews [Dataset]. Available at: <https://www.kaggle.com/datasets/saurav9786/amazon-product-reviews> (Accessed: 17 May 2026).
- Karau, H., Konwinski, A., Wendell, P. and Zaharia, M. (2015) Learning Spark: Lightning-Fast Big Data Analysis. Sebastopol, CA: O'Reilly Media.
- NVIDIA (2024) NVIDIA Quantum InfiniBand Platform. Available at: <https://www.nvidia.com/en-us/networking/quantum2/> (Accessed: 17 May 2026).
- OpenMP Architecture Review Board (2021) OpenMP Application Programming Interface, Version 5.2. Available at: <https://www.openmp.org/specifications/> (Accessed: 17 May 2026).